BÀI TẬP NHÓM GIỮA KỲ · NHÓM 10

# Cây trò chơi: Minimax và Alpha–Beta

Chọn đúng hành động, giải thích được nhánh cắt

**Giảng viên:** TS. Nguyễn Bích Vân – IAI–UET–VNU

**Học phần:** Biểu diễn tri thức và tìm kiếm nâng cao

**Nhóm 10: Nguyễn Hoàng Anh, Hoàng Trung Hiếu,Lê Bảo Ngân, Trần Nguyên Hiển Minh**

**Hạn nộp bài: hết ngày 27-10-2026**; ngày nhận đề là ngày 1 trong kế hoạch.

**Bảo vệ: 15–20 phút/nhóm.**

**Mã mẫu hoàn chỉnh, chạy được ngay.** Nhóm được dùng nguyên phần cài đặt, thay dữ liệu và chạy. Điểm đánh giá tập trung vào mô hình, trình bày thuật toán, tính tay, kiểm thử và nhận xét có minh chứng; không yêu cầu viết lại toàn bộ mã.

## Mục tiêu

Với cùng một cây trò chơi, vì sao có thể bỏ nhiều nhánh mà vẫn giữ được giá trị ở gốc? Khi nào số trả về chỉ là một cận? Vì sao kết quả có thể đúng về điểm nhưng sai về hành động? Nhóm trả lời bằng lập luận, bảng tính tay và chương trình.

## Phạm vi

Minimax và Alpha–Beta trên cây nhị phân hữu hạn, các tầng luân phiên MAX/MIN, utility ở lá đã cho. Không yêu cầu xây dựng luật cờ, giao diện đồ họa, tìm kiếm cục bộ, A* hoặc học máy. Các mục "Khảo sát nâng cao" là nhiệm vụ mở rộng về cận, tham số và kiểm chứng.

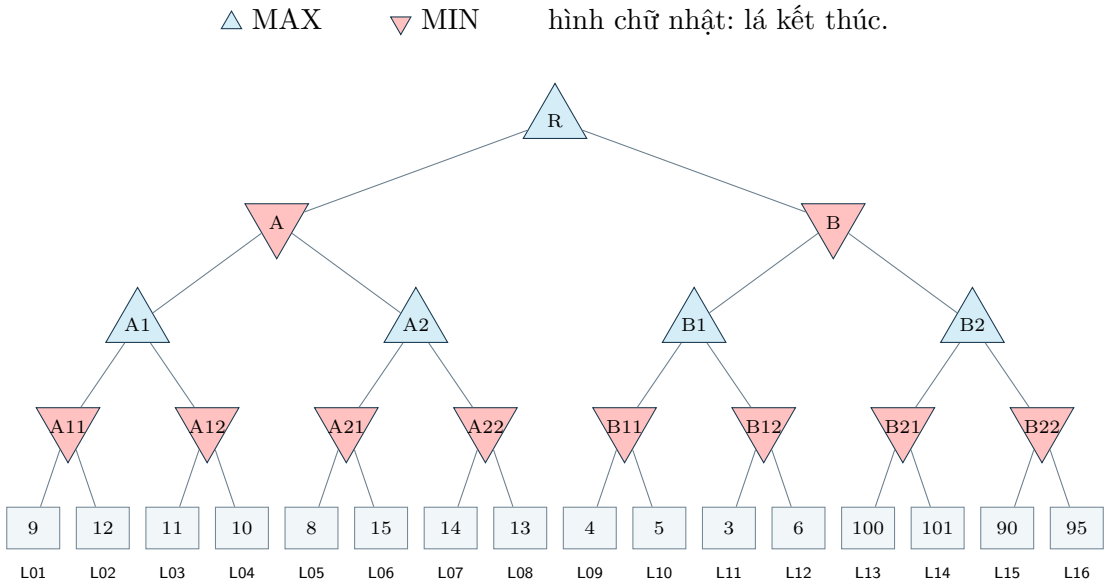
## Sản phẩm và thang điểm

| Mục | Nội dung | Điểm |
|---|---|---|
| A | Mô hình, trình bày hai thuật toán và điều kiện áp dụng | 15 |
| B | Tính tay, đường chơi, nhánh cắt và phân tích tham số | 20 |
| C | Chạy mã, thay dữ liệu, thực nghiệm chung và riêng | 20 |
| D | Kiểm thử và phân tích kết quả có minh chứng | 20 |
| E | Báo cáo, tái tạo và cộng tác bốn thành viên | 10 |
| F | Bảo vệ riêng từng thành viên | 15 |

Điểm mỗi sinh viên $= (G + F_i)/10$, với $G \leq 85$ là điểm bài nộp chung, $F_i \leq 15$ là điểm bảo vệ cá nhân. Bốn người có thể nhận điểm khác nhau. Tỷ trọng giữa kỳ trong điểm học phần được thông báo riêng.

## 1. Cây T0 và quy ước cố định

Ở mỗi nút, người ghi trên nút chọn **một trong hai con trực tiếp**. MAX muốn tăng utility; MIN muốn giảm cùng utility đó. Gốc R là MAX. Giá trị tại lá là lợi ích từ góc nhìn MAX, không phải chi phí cần cộng dọc đường.




**Dãy điểm lá từ trái sang phải:**

```
[9,12,11,10, 8,15,14,13, 4,5,3,6, 100,101,90,95]
```

Nhóm phải giữ nguyên T0. Tên L01 đến L16 gắn cố định với các lá trong hình; đổi thứ tự duyệt không đổi tên hay vị trí lá. Ví dụ A12 có hai con L03 và L04.

### Thứ tự duyệt và cách xử lý hòa

**LR**: con trái trước, con phải sau tại *mọi* nút. **RL**: thứ tự ngược lại tại *mọi* nút. Khi bằng nhau, giữ hành động đã chọn trước: cập nhật bằng $>$ ở MAX, $<$ ở MIN.

Alpha–Beta dùng cửa sổ gốc $(-\infty, +\infty)$, truyền cửa sổ mới nhất cho con tiếp theo, cắt cả khi bằng ngưỡng và trả giá trị đang giữ (*fail-soft*). Không dừng sớm theo một mức utility đặc biệt; không dùng cache hay gộp cây con.

> Không cộng các điểm trên đường. Phải phân biệt **giá trị Minimax thật**, **số trả về khi cắt** và **hành động được chọn**. Cây con chưa duyệt không được coi là đã biết điểm thật.

## 2. Phần A – Mô hình và trình bày thuật toán (15 điểm)

### A1. Mô hình bài toán [3 điểm]

Mô tả trạng thái, hành động, chuyển trạng thái, lượt đi và kết thúc trên T0. Vì sao chọn nhánh có lá 101 chưa chắc là tốt? Một đường từ gốc đến lá khác một chính sách ứng phó với đối thủ như thế nào?

### A2. Minimax [4 điểm]

Viết công thức truy hồi và mã giả đầy đủ cho lá, MAX và MIN. Giải thích khởi tạo $-\infty, +\infty$; làm rõ vì sao utility 0 không được coi là "chưa kết thúc". Đối chiếu mỗi khối mã giả với một đoạn trong `search.py`.

### A3. Alpha–Beta [5 điểm]

Trình bày vai trò của $\alpha, \beta$, thời điểm cập nhật, điều kiện cắt tại MAX/MIN. Giải thích hướng của cận khi trả sớm; vì sao phải giữ so sánh nghiêm khi đổi hành động. Cho biết kết quả nào ở gốc được bảo toàn khi so với Minimax trên cùng cây và cùng quy tắc hòa.

### A4. Giới hạn và độ phức tạp [3 điểm]

Với hệ số phân nhánh $b$, độ sâu $d$, nêu cận thời gian xấu nhất, tình huống Alpha–Beta không giảm số lá. Phân biệt bộ nhớ ngăn xếp đệ quy, cây đầu vào được lưu sẵn và log vết chạy; không khẳng định toàn chương trình chỉ dùng $O(d)$ bộ nhớ.

### Quy ước thống kê dùng chung cho cả bài

| Trường | Định nghĩa |
|---|---|
| `visited` | Số lời gọi đi vào nút, kể cả gốc và lá đã đến. |
| `expanded` | Số nút trong đã bắt đầu xét các con. |
| `leaves` | Số lá thực sự được đọc utility. |
| `cutoffs` | Số lần cắt có ít nhất một con còn lại chưa được gọi. |
| `pruned_branches` | Số cạnh tới con trực tiếp bị bỏ; không đếm toàn bộ cây con. |
| `unvisited_nodes` | Tên mọi nút không được gọi, gồm nút trong và lá. |
| `max_depth` | Độ sâu lớn nhất đã tới; gốc có độ sâu 0. |

Luôn kiểm tra `visited` = `expanded` + `leaves`. Với Minimax, hai số đếm cắt bằng 0. Gặp điều kiện cắt ở *con cuối cùng* không làm tăng số lần cắt vì không còn gì để bỏ.

## 3. Phần B – Tính tay trên T0 và tham số (16/20 điểm)

Làm tay trước khi chạy. Nộp `TinhTay.pdf` viết tay/bút điện tử; được dùng hình in sẵn để điền. Khi đối chiếu, ghi rõ chỗ đã sửa thay vì sửa ngược phần dự đoán cho khớp mã.

### B1. Minimax đầy đủ trên T0 [5 điểm]

Điền giá trị thật cho cả 15 nút trong, chỉ ra hành động tối ưu ở gốc. Đánh dấu một đường chơi khi cả hai bên đều chọn tối ưu, theo quy tắc hòa LR. Giải thích lựa chọn ở từng tầng; đếm số nút và số lá đã đọc.

### B2. Alpha–Beta từ trái sang phải [6 điểm]

Ghi thứ tự lá được đọc; tại mỗi lần cắt, ghi nút cắt, cửa sổ lúc cắt, số đang giữ, hướng cận và gốc cây con bị bỏ. Liệt kê riêng **các lá không đọc** và **toàn bộ các nút không thăm**; tính các số đếm đã quy định và tỷ lệ tiết kiệm lá so với B1.

Kiểm tra thật cẩn thận khi đi từ A sang B: có được khởi tạo lại $\alpha$ hay không? Các giá trị ở phần cây B2 có thể điền từ chính lần chạy Alpha–Beta này không? Muốn điền giá trị thật cho chúng, cần dùng kết quả nào?

### B3. Khảo sát nâng cao: thay L01 bằng $t$ [5 điểm]

Trong T0, thay riêng utility 9 tại L01 bằng $t \in [0, 20]$, giữ mọi điểm khác. Gọi cây mới là $T(t)$.

a) Lập biểu thức và chia khoảng cho giá trị Minimax ở gốc. Tìm hành động ở gốc và đường chơi tối ưu theo LR khi $t$ đi qua các mốc làm thay đổi lựa chọn.

b) Xác định điều kiện theo $t$ để L04 bị cắt khi chạy Alpha–Beta LR. Phân biệt ngưỡng đổi đường chơi với ngưỡng xuất hiện nhánh cắt mới.

c) Tính riêng $t = 10, 11, 14$: giá trị gốc, hành động, dãy lá đã đọc và số lá. Đối chiếu sau bằng Python.

> Câu tham số cần lập luận trên *cả khoảng*, không chỉ chạy vài số nguyên. Chương trình chỉ cần nhận $t$ nguyên để thực nghiệm; phần tính tay xét $t$ thực.

### Mẫu bảng vết cắt cần lập

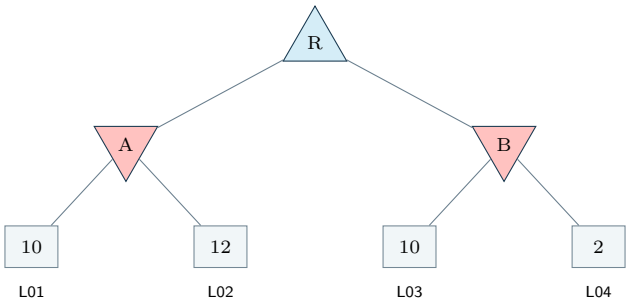
| Nút | Cửa sổ lúc cắt | Số trả | Gốc nhánh bỏ | Hướng cận |
|---|---|---|---|---|
| ... | $(\alpha, \beta)$ | $v$ | ... | $V \le v$ hay $V \ge v$ |

Một bất đẳng thức cận vẫn có thể đúng khi số trả tình cờ bằng giá trị thật; không được lấy sự trùng nhau ở một ví dụ làm bảo đảm chung.

## 4. Phần B4 – Đúng điểm nhưng sai hành động (4 điểm)

### Khảo sát nâng cao trên cây T1

Gốc MAX, hai con MIN. Các lá từ trái sang phải là $10, 12, 10, 2$. Cấu trúc và quy tắc chọn vẫn như mục 1.




a) Tính giá trị thật của A, B, R. Chạy Alpha–Beta LR, ghi cửa sổ vào B và phần bị cắt. Số B trả lên có phải giá trị thật không?

b) Một bạn thay điều kiện `score>best` tại MAX bằng `score>=best`. Giữ nguyên cách cắt. Chỉ ra *cả value và action* mà bản sai có thể trả ở R; nêu phản ứng của MIN chứng minh hành động ấy không tối ưu.

c) Viết mô tả một kiểm thử tự động phát hiện lỗi. Test chỉ so sánh value có đủ không? Có cần bỏ dấu bằng trong *điều kiện cắt* để sửa lỗi chọn hành động không?

### Dữ liệu T2: tình huống mọi điểm đều âm

T2 là cây nhị phân đầy đủ có ba cạnh từ gốc tới lá, gốc MAX. Dãy lá LR:

```
[-8,-4,-7,-2,-6,-3,-9,-5]
```

T2 dùng cho kiểm thử và thực nghiệm, không phải vẽ lại toàn bộ cây bằng tay. Dự đoán trước khi chạy: giá trị gốc có nhất thiết không âm không? Hậu quả của khởi tạo giá trị tốt nhất tại MAX bằng 0 là gì?

### Dữ liệu affine: cùng cây, đổi thang điểm

Với mỗi lá utility $u$ của T0, thay bằng $u' = 2u - 7$. Giữ nguyên cấu trúc và thứ tự. Dự đoán giá trị gốc, hành động và số lá được đọc của hai thuật toán. Kiểm tra rồi giải thích bằng tính bảo toàn thứ tự của phép biến đổi này; không suy tương tự cho phép nhân hệ số âm mà vẫn giữ MAX/MIN.

> Không cần cài thêm thuật toán khác. T1, T2, $T(t)$ và phép đổi thang điểm chỉ giúp kiểm tra chính Minimax/Alpha–Beta và cách báo cáo kết quả.

## 5. Phần C – Python và dữ liệu riêng (20 điểm)

### C1. Chạy và giải thích mã mẫu [6 điểm]

Chạy bản tối giản, rồi chạy `search.py` qua `run.py`. Chỉ rõ nơi kiểm tra lá, khởi tạo, gọi đệ quy, cập nhật hành động, truyền cửa sổ và cắt. Chế độ `--step` phải dừng ngay trong thực thi; tự chọn 3 sự kiện để dự đoán trước khi tiếp tục.

### C2. Ma trận chung: 24 cấu hình [6 điểm]

| Nhóm | Dữ liệu và thiết lập | Số dòng |
|---|---|---|
| Cơ bản | T0, T1, T2; hai thuật toán; LR/RL | 12 |
| Tham số | $T(t)$ với $t = 8, 10, 11, 12, 14$; hai thuật toán; LR | 10 |
| Đổi thang | T0 với $u \mapsto 2u - 7$; hai thuật toán; LR | 2 |

Dùng `pythonrun.py--batch--tagbat_buoc`. Mỗi dòng là một cấu hình, không phải một lần lặp. Lưu CSV thống kê và JSON đầy đủ. Đo thời gian là tùy chọn; khi so sánh, dùng cùng máy, tắt step/trace, thêm `--repeat3` để lấy trung vị.

### C3. Hai cây riêng, thêm 8 cấu hình [5 điểm]

N1: 16 lá, gốc MAX; N2: 8 lá, gốc MIN. Utility nguyên trong $[-20, 30]$, mỗi cây có ít nhất bốn giá trị phân biệt, khác ba cây chung. Tạo bằng `tao_du_lieu.py` với seed riêng, hoặc tự điền JSON; báo cáo dãy điểm thật đã dùng. Không đổi tên dữ liệu mẫu rồi coi là dữ liệu mới.

Chạy cả hai thuật toán theo LR/RL trên mỗi cây: $2 \times 2 \times 2 = 8$ dòng. Với N2, lập một phép tính Minimax từ lá lên gốc độc lập để kiểm tra kết quả; không dùng chính output đang kiểm làm kỳ vọng.

### C4. Tái tạo kết quả [3 điểm]

Lưu `output/bat_buoc.json/.csv` (24 dòng), `output/tu_tao.json/.csv` (8 dòng), một JSON trace của T0/Alpha–Beta/LR. Đổi thứ tự duyệt không đổi tên L01,... Ghi đúng lệnh và tệp để người khác tái chạy.

### Giao diện chính

```
root = make_tree(LEAVES, root_player="MAX")
result = solve(root, algorithm="alphabeta", order="LR",
               trace=True, step=False)
```

Bản Minimax có `principal_variation` để minh họa đường chơi đầy đủ. Bản Alpha–Beta để trường này bằng `None`: không ghép tùy tiện các hành động nội bộ có thể gắn với cận thành một đường tối ưu.

## 6. Phần D – Kiểm thử và nhận xét (20 điểm)

### D1. Ít nhất 8 kiểm thử riêng [8 điểm]

Mỗi thành viên phụ trách ít nhất **2 kiểm thử**; toàn nhóm có ít nhất **3 kỳ vọng tính tay độc lập**. Viết trong `test_nhom.py`; mỗi test phải có khẳng định, không chỉ in. Các test mẫu không thay cho tám test riêng.

Bao phủ ít nhất: utility 0; tất cả điểm âm; gốc MIN; hòa thật theo LR/RL; hòa giả giữa các cận ở T1; không sửa dữ liệu đầu vào; đẳng thức số đếm; trace bật/tắt không đổi kết quả. Có thể ghép thuộc tính liên quan vào một test. Tự bổ sung test số lá không hợp lệ hoặc ký tự sai kiểu.

Dùng `pythonchay_kiem_thu.py` để lưu log thật. Không xóa/comment test lỗi rồi báo tất cả đều đạt. Ghi lỗi đã gặp, sửa gì và test xác nhận.

### D2. So sánh hai thuật toán [8 điểm]

a) Đối chiếu value/action ở cùng cây, cùng thứ tự và cùng quy tắc hòa. Hai thuật toán phải thống nhất ở gốc; không đòi bảng giá trị trung gian có cùng ý nghĩa.

b) Tính tỷ lệ tiết kiệm lá $1 - L_{AB}/L_{MM}$ cho từng cặp tương ứng. Nếu đo thời gian, tách khỏi thời gian nhập Enter và ghi tệp. Không khẳng định nhanh hơn từ một lần đo trên cây rất nhỏ.

c) Tìm ví dụ trong các dữ liệu cho thấy đảo thứ tự có thể làm tốt hơn hoặc kém hơn. Không suy thứ tự LR hay RL luôn tốt nhất trên mọi cây.

d) Giải thích kết quả tham số và phép đổi $u \mapsto 2u - 7$; đối chiếu một cấu hình với tính tay. Nêu ít nhất hai giới hạn của khảo sát.

### D3. Phát hiện sai lầm [4 điểm]

Chọn hai trong ba lỗi: khởi tạo MAX bằng 0; thay action bằng so sánh không nghiêm; reset cửa sổ ở từng con của gốc. Với mỗi lỗi, chỉ dòng mã liên quan, dự đoán ảnh hưởng, nêu dữ liệu/test phát hiện. Có thể thử trên bản sao riêng `thu_loi.py`; không để lỗi trong chương trình chính.

**Bài nộp không chỉ là CSV.** Nhận xét cần chỉ rõ cây, cấu hình, con số và nguyên nhân. “Alpha–Beta luôn đọc ít lá hơn” không phải kết luận được chấp nhận nếu dữ liệu có trường hợp bằng nhau.

## 7. Tổ chức nhóm, bài nộp và bảo vệ

### Phân công cho đúng bốn thành viên

| Người | Chủ trì | Kiểm tra chéo |
|---|---|---|
| 1 | Mô hình, Minimax, tính tay B1 | Đọc vết Alpha–Beta |
| 2 | Alpha–Beta, B2 và T1 | Kiểm tra bảng Minimax |
| 3 | Dữ liệu, tham số, kiểm thử | Tái tạo kết quả của người 1–2 |
| 4 | Thực nghiệm, tích hợp và báo cáo | Kiểm tra JSON, test và lệnh chạy |

Mỗi người đều hiểu cả hai thuật toán, viết/giải thích ít nhất hai test và tham gia bảo vệ. Không giao một người chỉ ghép báo cáo. Họ tên/MSSV điền theo danh sách thực tế của nhóm hiện tại.

### Kế hoạch 10 ngày

Ngày 1–2 đọc đề, chạy ví dụ, phân công; ngày 3–4 làm tay song song với đọc mã; **ngày 5** đối chiếu T0/T1, có ít nhất 4 test. Ngày 6–7 hoàn thiện 8 test và chạy 24+8 cấu hình. Ngày 8–9 viết báo cáo, chuẩn bị slide, tập hỏi chéo. Ngày 10 giải nén chạy lại rồi nộp theo giờ thông báo.

### Phần E – Báo cáo và tái tạo [10 điểm]

Báo cáo **6–8 trang nội dung chính**, không tính bìa, phụ lục tính tay và mã; dùng mẫu Word hoặc LaTeX để điền. Trình bày thuật toán, tính tay–code, kiểm thử, kết quả, nhận xét và đóng góp. Không dán toàn bộ mã/trace dài vào báo cáo.

Nộp một `Nhom10_CayTroChoi_GiuaKy.zip` gồm `BaoCao.pdf`, `TinhTay.pdf`, `ThuyetTrinh.pptx` (6–8 slide), các tệp Python chạy được, `test_nhom.py`, `du_lieu_nhom.json`, `output/`, `README_NOP.md`, `DongGop.md`.

### Phần F – Bảo vệ cá nhân [15/người điểm]

**18 phút gợi ý:** 5 phút trình bày; 3 phút demo; 8 phút hỏi bốn người (khoảng 2 phút/người); 2 phút kết luận. Tổng nằm trong 15–20 phút. Mỗi thành viên có thể được hỏi bất kỳ phần nào.

Năm tiêu chí, mỗi tiêu chí 3 điểm: mô hình; dự đoán bước; chỉ đúng đoạn mã; xử lý thay đổi nhỏ; giải thích giới hạn.

> Trước nộp: giải nén vào thư mục khác, làm theo README từ đầu và kiểm các con số báo cáo khớp JSON/CSV. Mẫu báo cáo để trống không phải sản phẩm hoàn thành. Công cụ đóng gói chỉ kiểm thành phần, không đánh giá chất lượng bài.